

Benchmarking Table Extraction from Heterogeneous Scientific PDF Documents

Marijan Soric
DI ENS, ENS, PSL University, CNRS, Inria
Paris, France
marijan.soric@inria.fr

Ioana Manolescu
Inria, Institut Polytechnique de Paris
Palaiseau, France
ioana.manolescu@inria.fr

Cécile Gracianne
BRGM
Orléans, France
c.gracianne@brgm.fr

Pierre Senellart
DI ENS, ENS, PSL University, CNRS, Inria
Paris, France
pierre@senellart.com

Abstract

Table Extraction (TE) consists in extracting tables from PDF documents, in a structured format enabling automatic processing. While numerous TE tools exist, the variety of methods and techniques makes it difficult for users to choose the most appropriate one. We propose a novel benchmark for assessing end-to-end TE methods (from PDF to the final table) over 86k pages. We contribute an analysis of TE evaluation metrics, and a novel, rigorous evaluation process, which allows scoring each TE sub-task as well as end-to-end TE, and captures model uncertainty. Along with prior datasets, our benchmark comprises two new heterogeneous datasets of 39k samples. We run our benchmark on diverse models, including off-the-shelf libraries, tools, computer vision-based models and modern approaches using general and specialized vision language models. The results demonstrate that TE remains challenging: current methods suffer from a lack of generalizability when facing heterogeneous data, and from limitations in robustness and interpretability.

CCS Concepts

• **Information systems** → **Semi-structured data**; **Evaluation of retrieval results**; *Document structure*; *Data mining*; • **Computing methodologies** → **Object recognition**.

Keywords

Table extraction, Table detection, Table structure recognition

ACM Reference Format:

Marijan Soric, Cécile Gracianne, Ioana Manolescu, and Pierre Senellart. 2026. Benchmarking Table Extraction from Heterogeneous Scientific PDF Documents. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3770855.3817462>

KDD Availability Link:

The source code of the implementation accompanying this paper is available at <https://doi.org/10.5281/zenodo.20486310>.



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD '26, Jeju Island, Republic of Korea*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817462>

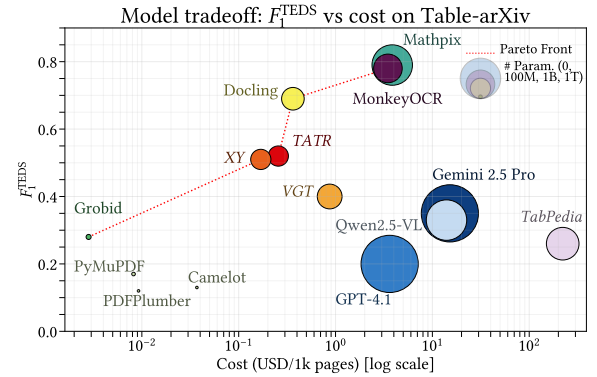


Figure 1: Performance vs. computational cost

1 Introduction

In PDF documents, such as scientific publications, business or administrative reports, etc., tables are often used to represent data in a way that is easy to read by humans. Such tables often contain valuable information, which cannot be easily found in other sources. Thus, retrieving the set of all tables from these documents is crucial in order to exploit such data. This is the goal of **table extraction** (TE, in short). TE is often a first step of data analysis pipelines, e.g., to answer questions over tables [5, 22] or combine them within a data lake [31, 33, 52].

Prior research has identified two core TE sub-tasks: **Table Detection (TD)** and **Table Structure Recognition (TSR)**, often studied and evaluated independently, each on its own dataset [10, 23, 37, 64]. However, in any real pipeline TD feeds TSR, so detection errors propagate downstream. Evaluating the two in isolation, on separate datasets (Figure 2), therefore fails to capture what we actually care about, which is why an *end-to-end* evaluation is needed. Moreover, new methods are typically compared only against others of the same family on homogeneous data, rather than across diverse approaches and document layouts.

In this paper, we address this gap by proposing carefully justified metrics for TE methods and, based on these metrics, comparing a large set of methods in an *end-to-end* fashion, on scientific PDF documents. We review existing methods on heterogeneous datasets (diverse in terms of layout and table styles), ranging from simple

rule-based libraries to object detection systems based on transformers, general and task-specialized Vision Language Models (VLMs), and a commercial tool. As modern approaches (based on vision language models) require heavy computation, we also investigate the tradeoff between performance and computational cost, as illustrated in Figure 1.

Our main contributions are as follows: (1) We compare academic methods to those widely used in practice, on four benchmark datasets. While prior work assessed TE subtasks in isolation, this paper enables end-to-end quality evaluation of TE methods. (2) We propose a new framework for TE evaluation, with formally justified, new end-to-end metrics; (3) We create and publicly share two novel datasets for TD, TSR, and TE, with high-quality ground-truth data.

We distinguish *text-based* PDFs, i.e., files produced via tools such as Office or $\LaTeX$, from *non-text* ones, which are mostly results of scan or photography. While the latter include historical (legacy) documents, the former are continuously created, in increasing numbers, e.g., on arXiv¹. Thus, in this work, we *focus on text-based PDFs only*. Non-text PDFs could be converted to text-based ones via Optical Character Recognition, however, this may introduce extra errors, whose impact we seek to avoid in our TE benchmark.

We define the TE, TD, and TSR tasks in Section 2, also discussing the relevant literature. We describe our datasets and the methods we compare in Sections 3 and 4, respectively. We present evaluation metrics and our evaluation methodology in Section 5. Experimental results are in Section 6. Our benchmark is archived at <https://doi.org/10.5281/zenodo.20486310>, together with additional material and an extended version with appendix.

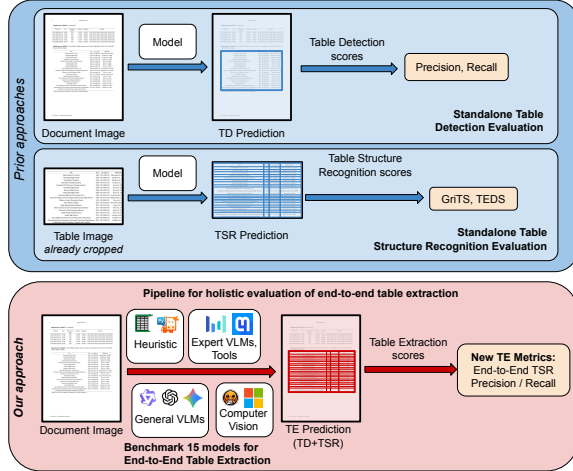


Figure 2: Prior approaches evaluate Table Detection and Table Structure Recognition in isolation, each on its own dataset and with TSR starting from gold-cropped table images. Our pipeline instead evaluates 15 end-to-end models on the same documents, feeding the TD output into TSR and scoring the result with dedicated end-to-end TE metrics.

2 Table Extraction

We view a **table** extracted from a document as a list of *rows*, each of which consists of a list of *cells*. Further, each cell has a *content*, that may be textual, numerical, mixed, etc. Note that different rows may have different numbers of cells; this is due to *merged cells*, i.e., cells spanning several rows and/or columns.

2.1 Table Extraction Tasks

Table Extraction (TE) involves detecting and recognizing a table’s logical structure and content from its unstructured presentation in a document. TE can be decomposed in subtasks, as follows.

Table Detection (TD). This consists in *detecting* tables in an input document (PDF document, or page given as a bitmap image). We assume tables are *aligned with the axes* of the pages, and might have been *rotated* by ± 90 degrees. Typically, a table is detected as a **rectangular bounding box**. Note that some TE models do not output bounding boxes, but only HTML tables.

Table Structure Recognition (TSR). This consists in *recognizing the structure of a table in terms of rows, columns, and cells*. The input can be a cropped table image or a page image with detected table bounding box. Table Content Recognition (TCR) *identifies the content (characters) within each cell*. Clearly, TCR depends on TSR for the identification of cells; the two tasks are often performed together. Thus, by a small abuse of language, we will denote *both tasks* by the TSR acronym. At the end of TSR, the table is fully captured in a structured format, and can be rendered in HTML with its rows, cells, spans, and content.

2.2 Prior datasets and benchmarks

The literature comprises several benchmarks for TD and TSR, the latter with or without TCR. Some datasets focus *solely on TD*, such as PubLayNet [79] and TNCr [1]. Others target *TSR only*, e.g.: SciTSR [10], focusing on scientific papers; PubTabNet [78], from biomedical articles; TabLeX [14], which uses $\LaTeX$ for dataset generation; SynthTabNet [47], a synthetically generated dataset; WTW [59] and iFLYTAB [73], from photos and scanned documents. Other datasets address *TD and TSR, but not TCR*, including: ICDAR-2019 [17], the dataset of a TD competition; TableBank [37], a larger dataset built using weak supervision from Word and $\LaTeX$ documents on the internet; PubTables-1M [63], the largest dataset available (one million tables). Some datasets support TE evaluation but *consist of only images (scans) of documents*, as opposed to PDFs with text. Most recent examples include TabRecSet [70] (real-world scanned documents), and OmniDocBench [50] (manuscript annotations, magazines). Among the few text-based PDF datasets is that from the ICDAR-2013 [23] TD competition. While FinTabNet [64, 77] (financial reports) meets our criteria, its poor data quality led us to exclude it from our study.

Most prior benchmarks evaluate only a *limited set of TE methods*, often focusing on introducing a new dataset rather than studying existing approaches [27]. In contrast, our benchmark evaluates a *diverse range of methods*, fifteen in total, from classical to very recent ones. In particular, we provide the first thorough comparison of

¹<https://arxiv.org/>

recent VLMs, proposed for document layout analysis, with specialized TE methods, on common datasets, with a unified comparison framework.

Existing benchmarks tackle *TD* and *TSR* as two independent problems and do not evaluate the quality of end-to-end TE methods. Consider a two-step TE method involving two models where the output of TD becomes the input of TSR model. *TSR benchmarks* start from detected (cropped) table images. In contrast, in *real-world deployments*, the quality of TSR output can strongly degrade if its input (TD output) is of poor quality. This impact is missed when evaluating TD and TSR separately; or, in reality, what matters is *end-to-end* TE result quality. This is why we need an **end-to-end benchmark**, and thus **new metrics** for evaluating the end-to-end model synergy.

Another limitation of existing benchmarks is that *they only consist of positive instances*, i.e., each sample includes at least one table. As we will show, this *inflates precision scores* and *compromises their relevance* for real-world performance.

Existing TE methods. An end-to-end TE model processes a document (or a page) to extract tables. Early heuristics methods relying on hard-coded patterns were limited to specific table types [9, 18, 28, 29, 57, 67]. Machine Learning (ML) later improved generalization [16, 26, 30, 32] but required heavy feature engineering, which was eventually surpassed by Deep Learning (DL) models [6, 60]. Thanks to larger training datasets, deeper networks and more efficient architectures, these now achieve superior accuracy and versatility. Notably, [25] used Convolutional Neural Networks (CNNs) for TD, and DeepDeSRT [61] was the first to use Faster R-CNN-based models [19] for TD and TSR.

Some models are focused on certain data types, for example scans of paper documents with distortion [45], others on robustness. More recently, expert VLMs specialized for TE or document parsing have emerged. We detail state-of-the-art methods in Section 4.

3 Our benchmark

Table 1: Datasets used in our benchmark (L: $\LaTeX$, W: Word, P: Professional publishing system, O: Other).

Dataset	% Type				Count		
	L	W	P	O	# Pages	# Positives	# Tables
PubTables	0	1	85	14	46 942	46 942	55 990
Table-arXiv	100	0	0	0	36 869	5 214	6 308
Table-BRGM	11	81	0	8	2 003	256	361
ICDAR-2013	?	?	?	?	238	135	163

Our benchmark for evaluating the quality of end-to-end TE methods is based on four datasets of *scientific* text-based PDF documents. These are structured documents primarily used in academic, research, or technical fields. Their tables vary widely in design (depending on the publisher), and domain, e.g., biomedical, mathematical, physical, geological, governmental (administrative), etc. Each dataset contains: **as input**, document pages (both as PDF files and as individual rasterized images) with coordinates of each word appearing within the page (such coordinates can be obtained

by standard PDF or image analysis libraries such as PDFAlto and PDFMiner²); **as output**, every table identified within each page with bounding box coordinates, as well as the table content and structure formatted as HTML markup. Table 1 shows dataset statistics with novel contributions in bold. We use the PDF metadata, extracted via pdfinfo, to classify document types. For ICDAR-2013, we only have access to processed PDFs (documents with deleted pages), which prevents inference of the document type.

PubTables-Test. First, we used a subset of the largest prior TD/TSR test dataset, PubTables-1M, from scientific biomedical articles in PubMed Central Open Access [56], enriched with the PDF files and ground truth tables as HTML markup. The documents tend to be fairly homogeneous in the way tables are typeset; the PDF metadata indicates that most documents were produced using professional publishing systems such as Adobe InDesign.

Table-arXiv. To ensure **heterogeneity** within our data, we have also automatically generated “Table-arXiv”, a new dataset of 37k samples built from the $\LaTeX$ source files of arXiv preprints. These sources were sampled from *the last 20 years* and *across all arXiv domains*. Using the $\LaTeX$ source code, we automatically generated TD and TSR (with TCR) annotations: for TD, by instrumenting the commands starting and ending a tabular environment, to add anchors that allow tracking their locations within the generated PDF; for TSR, we used LaTeXXML³ to generate HTML markup tables from the source file.

Table-BRGM. We have manually constructed and annotated the dataset “Table-BRGM”, ensuring high annotation quality, that is domain-specific and comprises French and English geological reports sourced from a selection of BRGM documents; BRGM is France’s reference public institution in Earth sciences. The documents contain geological survey reports and records. Its interest is that *its layout significantly differs from the others*, mainly because these were mostly produced with a Word processing software rather than a typesetting system such as $\LaTeX$.

ICDAR-2013. Finally, we also considered the standard ICDAR-2013 dataset, a well-known TE benchmark. This dataset, introduced at the 2013 ICDAR Table Competition, contains European Union and US Government PDF reports.

These datasets contain **heterogeneous** tables: small and large tables, tables with or without borders, empty cells, merged cells, etc., in English and French.

4 End-to-end methods

We tested various methods, ranging from simple Python libraries, to well-established extraction platforms, computer vision-based methods, general VLMs and expert VLMs. Table 2 summarizes the methods, grouped by type. Some methods (noted with “Conf.” in Table 2) output a confidence score (between 0 and 1) along with their predictions, indicating how confident the model is that a table exists in a given area. We refer to models that return a confidence as “probabilistic models” (even though the scores are not necessarily interpretable as probabilities). “BBox” and “HTML” state whether

²<https://github.com/kermitt2/pdfalto>, <https://github.com/pdfminer/pdfminer.six>

³<https://github.com/bruce-miller/LaTeXML>

Table 2: End-to-end methods for table extraction

Method	Type	Input	TCR	Output			Steps	#Param.	Usage
				BBox	Conf.	HTML			
Camelot	Baseline §4.1	PDF	PDFMiner	✓		✓	1	0M	CPU
PyMuPDF		PDF	PyMuPDF	✓		✓	1	0M	
PDFPlumber		PDF	PDFMiner	✓		✓	1	0M	
Grobid		PDF	PDFAlto	✓		✓	1	0.3M	
TATR	Computer Vision §4.2	Image	PDFAlto	✓	✓	✓	2	60M	CPU
XY+TATR		Image	PDFAlto	✓	✓	✓	2	60M	
VGT+TATR-struct		Image	PDFAlto	✓	✓	✓	2	280M	
Docling		PDF	EasyOCR	✓		✓	2	130M	
Qwen2.5-VL	General VLMs §4.3	Image	VLM	✓		✓	1	32B	GPU
GPT-4.1		Image	VLM	?		✓	1	?	API
Gemini 2.5 Pro		Image	VLM	✓		✓	1	?	API
TabPedia	Expert VLMs §4.4	Image	PDFAlto	✓		✓	2	8B	GPU
GOT		Image	VLM	✓		✓	1	0.7B	
MonkeyOCR		Image	VLM	✓	✓	✓	1	3B	
Mathpix	? §4.5	Both	?	✓		✓	?	?	API

the tool outputs a table’s coordinates (bounding box, **BBox**) and/or its structure in HTML. “#Param.” is the number of model parameters; “Usage” states how the tool is run (on CPU, GPU, or through an API).

4.1 Baseline Models

Commonly used Python libraries. We selected the most popular TE Python libraries on GitHub: PDFPlumber [62], PyMuPDF⁴ and Camelot⁵, with respectively (as of June 2026) 10.4k, 9.9k, and 3.7k stars. Taking text-based PDF as input, they have rule-based heuristics using the *lines* that are explicitly drawn or implied by the word alignment on the page for table detection and extraction.

Grobid. (GeneRation Of Bibliographic Data⁶ [43]) is a machine-learning library for extracting, parsing, and re-structuring raw documents such as PDF into structured XML/TEI-encoded documents, with a particular focus on technical and scientific publications; it is used in production, e.g., on France’s national preprint server HAL. Grobid structures the document’s text body into paragraph, section titles, reference, figures, *tables*, etc., which it represents as XML.

4.2 Computer Vision-based methods

More advanced methods for TE rely on computer vision techniques, taking images as input (not PDF). We study four end-to-end TE methods, with different architectures. Confidence scores are typically obtained from the final softmax layer of the neural network and are used to classify bounding boxes.

TATR. Table TRansformer (TATR) [63] is a model released by authors of PubTables-1M, pretrained on their dataset, using the DETection TRansformer (DETR) [8] architecture. It consists of two parts: a backbone model (CNN) which extracts image features, fed into an encoder–decoder transformer model for object detection tasks. In contrast with prior approaches, e.g. [41, 66, 81], DETR does not need to be given “anchors” (regions where to look for candidate objects to extract). Instead, DETR learns a small, fixed number of position encodings, so-called *object queries*, and uses them for extraction. TATR is composed of two models: one for

TD, *TATR-detect*, whose output feeds the *TATR-struct* model which performs TSR. Given a page as an image, TATR-detect performs object detection to locate tables. Then, in the cropped images, TATR-struct detects table structure elements. TATR-struct expects as input the locations of tokens (letters, digits, symbols, etc.) in the document as vertical and horizontal coordinates in pages; we use PDFAlto to extract them. Finally, we obtain a complete table, including textual cell content, and export it as HTML markup.

XY-cut+TATR. Next, we introduce a new probabilistic model we devised: XY-cut+TATR. We noted that TATR-detect was sometimes wrong on pages containing multiple tables: either some were missed, or, when recognized, their confidence scores were low. Object queries (decoder input) are positional embeddings learned during training, which contain abstract information about spatial locations (“where should the model look?”). Thus, each output, which is directly associated to an object query, is roughly specialized in an area. To improve TATR results, we pre-process each page given as input, by separating layout components that are isolated by white space; for this, we rely on the algorithm XY-cut [24]. XY-cut works as follows. By counting the number of black pixels⁷ along each axis, *X* and *Y*, we obtain *profiles of pixel distributions*, which are used to isolate rectangle components to form sub-images. Each sub-image is then fed as input to TATR-detect. This allows to help the model focus on individual areas. We will designate this model as XY.

VGT+TATR-struct. This model follows a pipeline similar to that of TATR, where TATR-detect is replaced with a multimodal vision component, Vision Grid Transformer [13] (VGT). This is a Document Layout Analysis tool; it produces structured representations of documents. In contrast with TATR, which only uses visual information, VGT also relies on textual information, which it views as a 2D grid. This model achieves state-of-the-art results on TD over the PubLayNet [80] dataset. VGT has two parts: a Vision Transformer, which models visual information, inspired by ViT [15] and DiT [36]; and a Grid Transformer (GiT), that models 2D language information with 2D token-level grid. Overall, the framework aims to generate better embeddings from the input image page, in order to feed the detection framework that performs object detections using 6 classes, including table. The detection framework is the Cascade R-CNN [7] detector, which is implemented based on the detection library Detectron2 [69].

Docling. Docling [3] is an advanced Python library for PDF document processing and understanding (60.8k stars on GitHub). It extracts tables in two steps: it leverages RT-DETR [75] for TD, and TableFormer [47] for TSR, while relying on EasyOCR⁸, a third-party OCR library, for TCR. Docling does not provide confidence scores for TD. RT-DETR, based on DETR, finds various layout components (table, text, title, etc.) in each page. RT-DETR adds an efficient hybrid encoder that processes multi-scale features (inspired by Deformable-DETR [82]); and the uncertainty-minimal query selection (inspired by DINO [71]) that improves the quality of initial object queries. This model has been re-trained on the DocLayNet dataset [55], comprising diverse data sources with varied layouts. TableFormer [47] is a vision-transformer model that identifies the

⁴<https://github.com/pymupdf/PyMuPDF>

⁵<https://github.com/camelot-dev/camelot>

⁶<https://github.com/grobidOrg/grobid>

⁷ Assuming PDF reports are written in black on a white background.

⁸<https://github.com/JaidedAI/EasyOCR>

structure of a table by using a custom structure token language [44], starting from an image of the table. Its architecture consists of an encoder (CNN backbone network and transformer encoder) producing features that represent the input image, and two transformer decoders: Structure Decoder and Cell BBox Decoder. The former generates the logical table structure as a sequence of tokens (HTML tags), while the latter predicts simultaneously bounding boxes of table cells.

4.3 General VLMs

General VLMs are large-scale multimodal models designed for broad applications, including document parsing. By integrating large OCR datasets during pretraining, models such as Qwen2.5-VL [4] have shown strong performance in document content extraction tasks.

We selected Qwen2.5-VL 32B, an open source model able to localize objects using bounding boxes, document parsing and “robust structured data extraction from tables”. We also use OpenAI’s GPT-4.1 [49] and Google’s Gemini 2.5 Pro [12] as proprietary models, since they are recent, popular, and reported to perform very well on multimodal tasks. Qwen2.5-VL and Gemini 2.5 Pro are expected to have visual capacities and spatial understanding because of their pretraining. It remains unclear whether GPT-4.1 supports object localization, but we maintained the same prompt across all VLMs for consistency. We use these general VLMs in zero-shot prompting: given an image and a prompt, the VLM outputs normalized table coordinates and HTML tables. As we will discuss in section 5.4, the model’s limited visual capacity poses a challenge for evaluation, since TD analysis typically requires table coordinates. API inference cost a total of 320 USD for GPT-4.1 and 1 200 USD for Gemini 2.5 Pro. Also, we cannot guarantee the absence of contamination: these VLMs may have been trained on the datasets from the benchmark.

4.4 Expert VLMs

Expert VLMs are large multimodal models specifically trained for document parsing tasks.

TabPedia. TabPedia [74] is a specialized VLM in visual table understanding, able to perform TD, TSR, table querying and table question answering. TabPedia is composed of High and Low Resolution Vision Encoders, using the Swin-B [42] and CLIP visual encoder ViT [58] architecture respectively. The pre-training stage aims to align the visual features to the large language model (LLM) Vicuna-7B [11], and the fine-tuning stage focuses on visual table-aware understanding. These pre-trained models have been finetuned on PubTables-1M, FinTabNet and PubTabNet for TD and TSR.

GOT. GOT-OCR 2.0 [68] is an “OCR 2.0” model, based on VLMs. GOT relies on three modules: an image encoder (ViT [38]), a linear layer, and an output decoder (OPT-125M [72] or Qwen-0.5B). It follows three training stages: a pre-training for the vision encoder, then a joint-training for the encoder-decoder and finally a post-training for the language decoder. The authors curated a dataset of tables by crawling a large collection of $\LaTeX$ source files from arXiv. GOT outputs a .tex file describing the whole page, including the tables, but without coordinates.

Monkey-OCR. MonkeyOCR-pro-3B [39] is a unified vision and language framework for robust document parsing. MonkeyOCR adopts a Structure-Recognition-Relation triplet paradigm, which simplifies the multi-tool pipeline of modular approaches while avoiding the inefficiency of using general VLM for full-page document processing. The first stage employs a YOLO-based [76] document layout detector; the second stage performs content recognition within detected regions through a LLM. In the final stage, the logical reading order of detected elements is inferred through relation prediction. The training dataset has not been disclosed.

4.5 Commercial tool

Mathpix. We evaluated Mathpix⁹, a closed-source commercial tool accessed via API. While its model architecture is not publicly disclosed, Mathpix is a STEM-specialized system rather than a general-purpose model. It focuses on scientific and technical document understanding (math, chemistry, tables, figures, and multi-column layouts). API inference cost a total of 310 USD.

5 Metrics

In this section, we present metrics used for TD, TSR, and TE, as well as our evaluation methodology.

We first introduce some notations. From now on, we *highlight the first occurrence of each notation encasing it in a box*, for quick reference. Let $\boxed{\mathcal{P}}$ be the set of TE model outputs: each of its elements is a tuple that includes a bounding box, a predicted HTML table $\hat{y}$, and, if available, a confidence score c_{table} . When the models lack a confidence score, we consider $c_{\text{table}} = 1$ in all $\mathcal{P}$ entries. We define $\boxed{\mathcal{P}^+} \subseteq \mathcal{P}$ as the set of positive predictions: those that we consider to be actual tables. For probabilistic models, where confidence scores can be interpreted as table probabilities, we use a threshold $\boxed{\theta_c}$ to define positive predictions. This threshold draws a decision boundary: only predictions with scores above it are counted as positive. In this case, we set $\boxed{\mathcal{P}_{\theta_c}^+}$ to be $\{(\hat{y}, c_{\text{table}}) \in \mathcal{P} \mid c_{\text{table}} > \theta_c\}$, with confidence score threshold $\theta_c \in [0, 1]$. For the non-probabilistic models, $c_{\text{table}} = 1$, and thus we have $\mathcal{P}^+ = \mathcal{P}$.

5.1 Table detection (TD)

TD performance can be evaluated using traditional metrics inspired from Information Retrieval. Specifically, a True Positive (TP) is a table correctly recognized, a False Positive (FP) is a table found by the model where a human user would not consider a table exists, and a False Negative (FN) is a table recognized by human users but missed by the model. Given a table predicted by the model, and a table in the gold standard, to identify TPs, we rely on the Intersection-over-Union (IoU) metric between the areas of the prediction, and the gold standard tables. If IoU is above a given threshold $\boxed{\theta_J} \in [0, 1]$, the prediction is counted as a TP. For a given TD method, $\boxed{\mathcal{P}^{++}} \subseteq \mathcal{P}^+$ denotes the set of true positive predictions.

Average precision. For probabilistic models, the set of positive predictions $\mathcal{P}^+ \in \{\mathcal{P}_{\theta_c}\}_{\theta_c}$ depends on the choice of a confidence score threshold θ_c and a IoU threshold θ_J , thus we obtain a set of corresponding scores $\{X_{\theta_c, \theta_J}\}_{\theta_c, \theta_J}$, where X may denote Precision,

⁹<https://mathpix.com/>

Recall, etc. To aggregate over several θ_c values, in the object detection literature, the Average Precision metric is used [40]. The Average Precision at IoU threshold θ_j is the area under the *Precision–Recall curve* for a fixed θ_j , varying θ_c . By ranking predictions in the increasing order of their confidence score, we can compute Precision and Recall, at confidence score θ_c (where the positive predictions are elements from $\mathcal{P}_{\theta_c}^+$) and at matching threshold θ_j , which leads to pairs of Precision–Recall (the curve). The benefit of the AP_{θ_j} metric is to account for the entire recall range, not just a single recall threshold. We compute it using Scikit-learn [54].

Estimating model confidence via its (mis)calibration. The metrics used above (X_{θ_c, θ_j} , and AP_{θ_j}) rely on binary decisions, where each prediction is either positive or negative. We seek a finer-grain evaluation, accounting not only for these binary choices, but also for the associated confidence scores. A probabilistic model is said to be **calibrated** when its probabilities (confidence scores) are aligned with the real-world outcomes [21]. A calibrated model is desirable because its predictions can be seen as more trustworthy. Formally, model calibration is defined by: $\mathbb{P}[\hat{Y} = Y \mid \hat{P} = p] = p$ where $[\hat{Y} = Y]$ is the event “the model prediction is right” and $[\hat{P} = p]$ is the event “the confidence score (seen as a RV) is equal to p ”, for $p \in [0, 1]$. If this is the case, the random variable $\hat{P}$ perfectly reflects the prediction trustworthiness. Based on this definition, the miscalibration of a classifier can be measured by the *expected calibration error (ECE)* [46].

However, in the case of object detection, we cannot enumerate $[\hat{Y} = Y]$, but only $[\hat{Y} = Y = 1]$, because there is a multitude of negative locations (possible bounding boxes that do not contain a table). Thus, for an object detection model, calibration is measured with respect to the precision [34], leading to *the object detection version of ECE*, or **D-ECE**, in short: $\text{D-ECE} = \mathbb{E}_{\hat{P}} \left[\left| \mathbb{P} \left[\hat{Y} = Y = 1 \mid \hat{P} = p \right] - p \right| \right]$. To compute D-ECE efficiently, we approximate (discretize) by partitioning predictions into equal bins, and taking a weighted average of the bins’ (Precision–Confidence) gap.

5.2 Table structure recognition

Once a table has been correctly detected (TD yielded a true positive, TP), we want to evaluate TSR result quality, by checking that the predicted table has the correct table shape (topology, structure), and that cells have the expected content.

A naïve approach for TSR evaluation could be to tackle it as an object detection task by comparing predicted bounding box cells with Ground Truth (GT) cells by their absolute positions. This approach is vulnerable to *shifts*: a slight shift along one or both coordinates of a predicted bounding box cell would disproportionately penalize the models. Therefore, we prefer metrics that rely on the structure, not on absolute coordinates. The 4-gram BLEU score [51] (BLEU-4) metric compares predictions and GT HTML tables as strings with markup structure (tags) and content but lacks spatial structure evaluation, and needs exact matches for cells. The DAR metric [20] (Directed Adjacency Relation) is based on immediate neighbors, i.e., the relative positions of non-empty cells. As shown in [78], DAR cannot detect errors caused by empty cells and misalignment of cells beyond immediate neighbors (it does not capture the global 2D structure) and uses exact match. Thus,

we chose to use two metrics: GriTS [65], which evaluates tables directly in their matrix form and computes a similarity between these matrices, and TEDS [78] which compares tables seen as trees of HTML tags. We describe these metrics below.

TEDS. The metric Tree-Edit-Distance-Based Similarity (TEDS) measure views each HTML table as follows. There is a root `<table>` with children `<tr>` (nodes are table rows). The leaves of the tree are cells `<td>` with attributes `colspan`, `rowspan` and content.

TEDS builds upon the previous Tree-Edit Distance [53], in turn inspired by the Levenshtein Distance [35]. The latter counts the minimum number of insertions, deletions, and substitutions to transform a string into another string. Similarly, TEDS counts the node deletions, insertions, relabeling and content modifications needed to transform a tree (HTML table) into another, as follows: the cost to *insert* or *delete* a node is 1; the cost of *relabeling* a cell content is the normalized Levenshtein similarity between the two strings. Formally, for trees T_p, T_{GT} : $\text{TEDS}(T_p, T_{GT}) = 1 - \frac{\text{EditDist}(T_p, T_{GT})}{\max(|T_p|, |T_{GT}|)}$ where $|\cdot|$ is the number of nodes.

GriTS. The Grid Table Similarity (GriTS) family of metrics considers table structure from three perspectives: *Topology*, describing the rows and columns each cell spans over, in a two-dimensional grid; *Content*, which refers to the textual content within each cell; and *Location*, denoting the rectangular span in a pixel matrix that is occupied by each cell. This leads to *three GriTS similarity metrics*. We ignore the one based on Location (coordinates) since it suffers from the shift problem described above. Each metric takes as input two matrices of cells, the prediction and the GT, and computes their *2-dimensional Longest Common Subsequence* (2D-LCS, in short). First, tables are represented as matrices P, G (prediction, GT) entries in E , which is $\mathbb{Z}^4$ and V^N for the Topology metric and Content metric, respectively. From (P, G) , we extract *the two substructure matrices* $\tilde{P}$ and $\tilde{G}$ (of the same shape between themselves) *that are the most similar according to f* , the penalty function between the grid cells’ properties. The *penalty function* f depends on the sub-task (Topology, Content), as explained below. With $|\cdot|$, the matrix size (number of grid cells): $\text{GriTS}_f(P, G) = 2 \frac{\sum_{i,j} f(\tilde{P}_{i,j}, \tilde{G}_{i,j})}{|P| + |G|}$. The *entry-wise penalty function* $f: E \times E \rightarrow [0, 1]$ measures the *partial correctness* between two entries with the same coordinates; it replaces the 0 or 1 value used for the vanilla 2D-LCS computation [2]. Concretely, f is IoU for Topology, and LCS for Content.

HTML markup vocabulary to be used in our evaluation. One last aspect needs to be settled with respect to the metrics based on HTML markup (TEDS and GriTS). To avoid being penalized by different sets of HTML tags used by different models, we normalize each output preserving only three tags: `<table>`, `<tr>`, `<td>`. From now on, we implicitly consider that each TSR metric applies over such normalized outputs (and normalized GT) only.

5.3 Table extraction

We aim at evaluating TE to compare directly end-to-end methods as a whole. To this end, we propose a metric similar in spirit with the well-known Precision and Recall metrics, because these are easily interpretable. Specifically, we replace *binary* values (1 for TP and 0 for FP) by *continuous scores, parameterized by the TSR score*, which

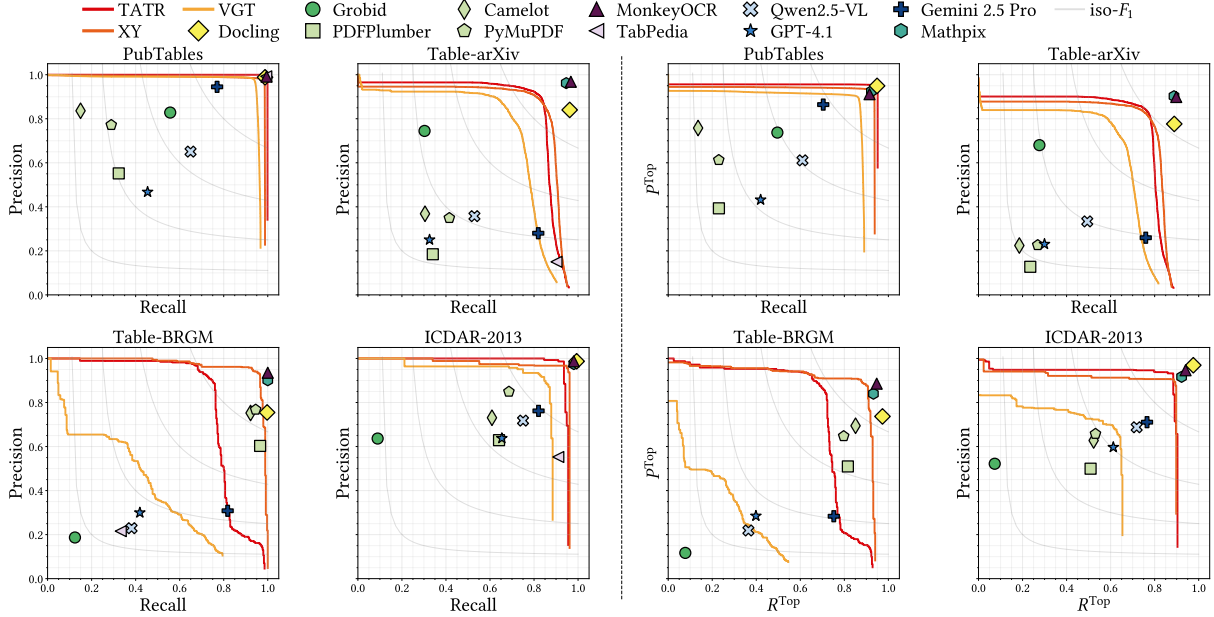


Figure 3: (left) Precision–Recall for *bbox* TD; (right) $P^{\text{Top}} - R^{\text{Top}}$ for *bbox* TE

measures how accurately table structure and content are extracted. Recall that θ_J is the IoU threshold above which table detection is considered to have found a table.

Definition 1 (TSR Precision and Recall). Denoting for each detected (positive) table i its IoU J_i , given a TSR metric which associates to each positive i the score s_i^{TSR} , we define the TSR precision and recall as:

$$P^{\text{TSR}} = \frac{1}{|\mathcal{P}^+|} \sum_{i \in \mathcal{P}^+} s_i^{\text{TSR}} \cdot \mathbf{1}_{[J_i > \theta_J]} \quad R^{\text{TSR}} = \frac{1}{|\mathcal{G}|} \sum_{i \in \mathcal{P}^+} s_i^{\text{TSR}} \cdot \mathbf{1}_{[J_i > \theta_J]}$$

where $\mathbf{1}_{[J_i > \theta_J]}$ is the indicator function whose value is 0 for tables whose IoU is below θ_J and 1 for the others and $|\mathcal{G}|$ is the number of ground-truth tables.

Intuitively, in the metrics defined above, we propagate the strength (or confidence) of the table detection, in order to also reflect it in the “downstream” task of TSR.

5.4 Evaluation Methodology

We now explain how we apply these metrics to ensure a fair comparison of all models and to evaluate end-to-end TE methods.

Text-based Table Detection. For **methods that output table locations** (bounding boxes), we use the metrics presented in Section 5.1, based on IoU. For models which **do not output bounding boxes** (GOT, Table 2), we replace the IoU by a Jaccard index (over tables content) to determine whether a table has been detected, based only on the predicted and GT tables HTML markup. Given the *multisets* of 2-grams obtained from the text content of a predicted table (S_P), respectively, from the GT (S_{GT}), we define: $\text{Jaccard}(S_P, S_{GT}) = \frac{|S_P \cap S_{GT}|}{|S_P \cup S_{GT}|}$ where $\cup$ is the multiset union (retaining each multiset element with its maximum multiplicity),

and $\cap$ the multiset intersection (based on minimum multiplicity). Accordingly, we call “*bbox* TD” and “*txt* TD” the TD evaluations based on bounding boxes, respectively, table content. After the TD evaluation, TSR evaluation is carried over the true positives $\mathcal{P}^{++}$.

Evaluating TSR impacted by TD. Traditionally, TD and TSR are evaluated independently. However, in downstream tasks, models are used sequentially to perform end-to-end TE, thus TSR evaluation depends on TD. Note that most recent models tested are one-step TE methods, and we may not be able to separate (discern) the two sub-models. Aiming for an end-to-end evaluation, we choose not to use a predefined dataset as input for TSR. Instead, we use the set of true positives $\mathcal{P}^{++} \subseteq \mathcal{P}^+$ obtained from the TD inference part; we use $\theta_J = 0.5$ to distinguish TP from FP. In this way, the TSR score better reflects what can be observed in the use of the end-to-end model. The final TSR score is then obtained by averaging the scores over the set of TP. This has two consequences: (i) *TSR evaluation is not independent of TD*; (ii) *We can no longer directly compare TSR results between models*, since the input dataset $\mathcal{P}^{++}$ is different for each model (determined by the model’s performance on TD). Consequently, if TD fails to correctly¹⁰ detect tables, the structure model will be penalized. While this may seem “unfair” to TSR models, it reflects *the reality of end-to-end TE result quality*, which are measured on the final results.

6 Experimental evaluation

We begin by analyzing TD results (subsection 6.1), comparing results obtained with *bbox* and *txt* TD, model calibration, and the tradeoff between performance and computational cost. Next, we

¹⁰For instance, a TP with IoU at 51% may miss part of the table’s content.

Table 3: AP scores for $bbox$ TD and Average AP^{TSR} for $bbox$ TE across datasets for probabilistic models.

Method	PubTables				Table-arXiv				Table-BRGM				ICDAR-2013			
	TD		TE metrics ($bbox$)		TD		TE metrics ($bbox$)		TD		TE metrics ($bbox$)		TD		TE metrics ($bbox$)	
	AP	AP^{Top}	AP^{Con}	AP^{TEDS}	AP	AP^{Top}	AP^{Con}	AP^{TEDS}	AP	AP^{Top}	AP^{Con}	AP^{TEDS}	AP	AP^{Top}	AP^{Con}	AP^{TEDS}
TATR	1.00	0.91	0.80	0.80	0.86	0.73	0.56	0.52	<u>0.82</u>	<u>0.73</u>	<u>0.67</u>	<u>0.64</u>	0.96	0.81	0.67	0.66
XY+TATR	<u>0.98</u>	<u>0.88</u>	<u>0.78</u>	<u>0.78</u>	0.86	0.73	0.56	<u>0.51</u>	0.95	0.87	0.78	0.73	0.96	<u>0.79</u>	<u>0.64</u>	<u>0.63</u>
VGT+TATR-struct	0.96	0.81	0.69	0.70	0.75	0.60	0.44	0.40	0.46	0.21	0.15	0.14	0.88	0.51	0.33	0.34

Table 4: F_1 -scores for $bbox$ and txt TD and F_1^{TSR} -scores for $bbox$ TE across datasets for models.

Method	PubTables					Table-arXiv					Table-BRGM					ICDAR-2013				
	TD		TE metrics ($bbox$)			TD		TE metrics ($bbox$)			TD		TE metrics ($bbox$)			TD		TE metrics ($bbox$)		
	F_1^{txt}	F_1^{bbox}	F_1^{Top}	F_1^{Con}	F_1^{TEDS}	F_1^{txt}	F_1^{bbox}	F_1^{Top}	F_1^{Con}	F_1^{TEDS}	F_1^{txt}	F_1^{bbox}	F_1^{Top}	F_1^{Con}	F_1^{TEDS}	F_1^{txt}	F_1^{bbox}	F_1^{Top}	F_1^{Con}	F_1^{TEDS}
Camelot	0.23	0.25	0.23	0.20	0.20	0.11	0.33	0.20	0.15	0.13	0.75	0.83	0.76	0.68	0.68	0.54	0.66	0.57	0.51	0.50
PyMuPDF	0.31	0.42	0.33	0.29	0.27	0.13	0.38	0.25	0.19	0.17	0.63	0.85	0.71	0.64	0.62	0.55	0.76	0.59	0.51	0.49
PDFPlumber	0.30	0.41	0.29	0.25	0.22	0.09	0.24	0.17	0.13	0.12	0.52	0.74	0.63	0.56	0.54	0.46	0.63	0.50	0.43	0.41
Grobid	0.61	0.67	0.59	0.51	0.47	0.34	0.43	0.39	0.31	0.28	0.12	0.15	0.09	0.07	0.06	0.15	0.16	0.13	0.12	0.11
Docling	0.96	0.99	0.95	0.86	0.86	0.70	0.90	<u>0.83</u>	<u>0.72</u>	0.69	0.78	0.86	0.84	<u>0.80</u>	0.79	<u>0.98</u>	0.99	0.97	0.90	0.90
Qwen2.5-VL	0.90	0.65	0.61	0.52	0.54	0.50	0.40	0.38	0.31	0.31	0.69	0.29	0.27	0.24	0.24	0.90	0.73	0.70	0.64	0.64
GPT-4.1	0.68*	0.46*	0.42*	0.35*	0.36*	0.29	0.28	0.26	0.20	0.20	0.43	0.35	0.33	0.27	0.27	0.65	0.65	0.60	0.53	0.53
Gemini 2.5 Pro	0.85*	0.78*	0.71*	0.66*	0.69*	0.27	0.36	0.34	0.30	0.30	0.45	0.45	0.41	0.38	0.39	0.79	0.79	0.74	0.70	0.71
TabPedia	–	<u>0.91</u>	TO	TO	TO	–	0.22	TO	TO	TO	–	0.26	TO	TO	TO	–	0.69	TO	TO	TO
GOT	0.39	–	–	–	–	0.14	–	–	–	–	0.07	–	–	–	–	0.28	–	–	–	–
MonkeyOCR	0.96	0.99	0.91	<u>0.85</u>	<u>0.87</u>	0.67	0.97	0.90	0.79	<u>0.78</u>	<u>0.82</u>	0.97	0.92	0.86	0.84	0.99	<u>0.98</u>	<u>0.94</u>	<u>0.88</u>	0.87
Mathpix	<u>0.93</u>	0.99	<u>0.92</u>	0.84	0.88	<u>0.68</u>	<u>0.95</u>	0.90	0.79	0.79	0.90	<u>0.95</u>	<u>0.88</u>	0.78	<u>0.82</u>	0.95	<u>0.98</u>	0.92	0.84	<u>0.89</u>

analyze TE results (subsection 6.2) using our new end-to-end evaluation framework. Our code is archived on Zenodo. Experimental results will answer the following questions raised previously: (Q1) Which models demonstrate the most consistent performance across all datasets? (Q2) Do low-cost models perform well in some cases? (Q3) Are VLMs reliable and competitive? (Q4) Which models offer the best tradeoff between performance and computational cost across datasets? (Q5) Are confidence scores trustworthy?

6.1 Table detection performance

Figure 3 (left) plots Precision–Recall curves from $bbox$ TD over our four datasets. Models are distinguished by colors (same as in Table 2). We plot *probabilistic model scores as curves* and represent the *other models as dots*, using IoU threshold of $\theta_I = 0.5$. Moreover, we draw iso- F_1 curves in gray (F_1 values: 0.2, 0.4, 0.6, 0.8). The closer a curve is to the top-right corner, the better its trade-off between Precision and Recall. Finally, we compute Average Precision scores, denoted as $\overline{AP}$, which is the area under the Precision–Recall curve (see Table 3) to easily compare curves. Table 4 gathers F_1 -scores for other models with txt and $bbox$ TD (denoted $\boxed{F_1^{txt}}$, $\boxed{F_1^{bbox}}$). We use the former to compare GOT with other models. For Gemini and GPT, scores with an asterisk (*) are for results obtained using a subset of the data (40k over 47k images), as inference was stopped after 1 500 USD due to budget constraints. The best results are shown in bold, and the second best are underlined.

Having four datasets allows us to observe different behaviors of models depending on the document layout and table styles. Indeed, models’ performance varies when changing the datasets. For example, Grobid, trained on scientific documents, has low F_1 -score on

Table-BRGM and ICDAR-2013 but significantly higher on PubTables. In contrast, heuristic methods perform well on Word-typeset tables with borders (Table-BRGM), but struggle on $\LaTeX$ -typeset tables (Table-arXiv). Among probabilistic models, XY achieves better performance on Table-BRGM, but does not maintain its lead over the other three datasets. On our PubTables dataset, TATR obtains an almost perfect score because it has been pretrained on PubTables-train, and our test set has similar inputs (in terms of layout or table style). VGT always outperforms TATR and XY. The Docling, MonkeyOCR and Mathpix methods, relying on distinct approaches (object detection and expert VLM for the former two) share the best performance on our benchmark with almost perfect F_1^{bbox} -scores (Q1). Since we noticed that MonkeyOCR outputs only perfect confidence scores, we decided to use $\mathcal{P}^+ = \mathcal{P}$, which is why it is compared to other non-probabilistic models.

GOT generates unnecessary nested tables, and its table content is misaligned with the ground truth, resulting in poor F_1^{txt} -scores; its heuristics are unreliable. TabPedia performs well on PubTables (it has been pre-trained on its train set), detects most tables in Table-arXiv and ICDAR-2013 (high recall), but generates many false positives (low precision). We interrupted TabPedia experiments during the TSR phase due to timeout (TO), exceeding allocated GPU hours. A full inference would have resulted in computational costs an order of magnitude higher than the second most expensive model (Figure 1).

On our two new datasets, Table-arXiv and Table-BRGM, the precision of general VLMs falls significantly. These datasets include negative samples, as they comprise real scientific documents which do not contain tables on every page (see Table 1). Since VLMs *almost always generate a prediction for each page*, this results in high number of FP, while their recall remains high (Q3). Such behavior

is not observable on a dataset like PubTables, which consists solely of pages containing tables.

Depending on the layout and table styles, for example on Table-BRGM, *heuristic methods* such as PyMuPDF, Camelot *compete with computer vision-based approaches and even outperform VLMs* (GPT, Gemini, TabPedia, GOT) *for a much lower computational cost* (Figure 1)¹¹ (Q2). To estimate costs, we tracked CPU and NVIDIA A100 GPU time and applied equivalent cloud computing rates from AWS¹². This enables a cost comparison with proprietary general VLM APIs. Note that XY is faster than TATR because it processes fewer potential table images during the TSR phase. The Pareto front depends on the dataset, but it is mainly composed of Grobid, PyMuPDF, XY, TATR, Docling, MonkeyOCR and Mathpix (Q4).

As expected, results obtained through *bbox* TD (Table 4) tend to be better than those with *txt* TD: detecting a table by its bounding box is easier than by its content because alignment between ground truth and model outputs (e.g. formula alignment) is a challenge in text-based evaluation. However, for Qwen2.5-VL and GPT-4.1, the opposite is observed: they manage to correctly extract tables, but output approximate table coordinates (Q3).

Figure 4 illustrates miscalibration for probabilistic models on Table-arXiv. Reliability diagrams [48] are histograms showing *precision as a function of confidence*. For perfect calibration, the histogram should match the identity function (the hatched histogram). Thus, the gap between the colored and identity histograms denotes a model’s miscalibration (the lower, the better); D-ECE (Section 5.1) measures this. Figure 4 shows that TATR is a poorly calibrated model (binary confidence scores make its confidence scores become meaningless) compared to VGT (Q5). For example, among predictions with confidence scores between 80% and 90%, on average, less than 2% are actual TP. The poor calibration of XY+TATR (D-ECE=0.45) comes from data drift: the model (TATR-detect) was trained on full-page images but applied to much smaller cropped page images. Raw TATR itself is already poorly calibrated, and this issue is compounded in XY+TATR. Post-hoc recalibration is an option but is dataset-dependent and does not generalize reliably. However, we found that TATR is calibrated on PubTables (D-ECE=0.03), which is close to its training set.

From these experiments, we draw the following lessons.

There are *two paradigms for TE probabilistic models usage* in downstream tasks. First, one can consider only positive predictions $\mathcal{P}^+$, which requires setting a confidence threshold. Second, we can consider the whole set of detected tables $\mathcal{P}$, with their attached confidence scores. In the first case, TATR fits perfectly, and the θ_c choice is easy according to Figure 4 (for example $\mathcal{P}^+ = \mathcal{P}_{0.9}$). Docling and MonkeyOCR also fit this case, without choosing any threshold. In the second case, VGT ensures interpretable and reliable confidence scores (Q5).

6.2 Table extraction performance

Figure 3 (right) illustrates Precision–Recall weighted by the TSR scores GriTS Topology (see subsection 5.3) from *bbox* TD. We compute $P^{\text{Top}} - R^{\text{Top}}$ with $\theta_J = 0.5$. In Table 3 and 4 the “TE metrics”

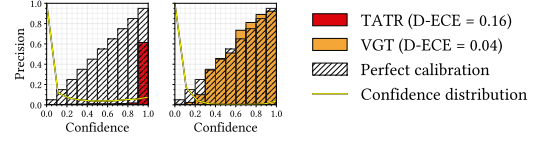


Figure 4: Reliability diagrams on Table-arXiv.

columns summarize F_1 and AP metrics for different TSR scores (denoted $\boxed{AP^{\text{TSR}}}$, $\boxed{F_1^{\text{TSR}}}$). Though scores decrease for metrics with TSR, regardless of the TSR score used, the ranking remains almost the same across methods.

VLMs like GPT suffer from table content hallucination, with performance dropping from $F_1^{\text{bbox}} = 0.35$ to $F_1^{\text{Con}} = 0.27$ on Table-BRGM (Q3). GriTS Topology scores are closer to vanilla $P-R$, which means that when a table is detected (with $\text{IoU } \theta_J = 0.5$), most of the time, the table topology (rows, columns, spanning cells) is correctly found. However, Precision and Recall with TEDS are lower, because they take into account partial token match. We obtain similar results using GriTS Content. TE scores fall for all models, revising the initial positive assessment of task TD, e.g. MonkeyOCR and Docling (Q1).

6.3 Experiment conclusions

Method performance varies significantly depending on dataset style, a dependency captured by our diverse benchmark datasets. This variability arises from intrinsic design choices: either hard-coded features (for heuristic-based methods) or pretraining data (for probabilistic models). Despite their scale, general VLMs are outperformed by smaller and cheaper, expert VLMs (MonkeyOCR) and computer-vision models (Docling, TATR) across datasets for TE. Confidence scores do not always provide interpretable insights, yet calibration matters in practice for usability: VGT stands out positively in this regard, while TATR and XY+TATR do not. For TSR, TCR remains challenging in terms of content accuracy. Overall, while Docling, MonkeyOCR and Mathpix (the most promising TD models) demonstrate strong performance, they have yet to achieve perfect TE (e.g., $F_1^{\text{TEDS}} < 90\%$). Our benchmark enables direct, end-to-end comparison of methods across heterogeneous data and from multiple perspectives (accuracy, cost) at a glance.

7 Conclusion

We compared different methods for end-to-end table extraction from scientific PDF documents. Our main conclusion is that table extraction *remains a challenging problem* (both table detection and table structure recognition). Our new framework for TE allows better end-to-end evaluation, encompassing TD and TSR, over a wide variety of model types on new datasets (Table-arXiv and Table-BRGM). This provides novel resources for advancing TE research and development.

Acknowledgments

This work was supported by Inria and BRGM via the GéolAug project, and was performed using HPC resources from GENCI-IDRIS (Grant 2025-AD010615780R1). We are grateful to the CLEPS infrastructure from Inria Paris for providing resources and support.

¹¹We chose to display the best F_1 -score for probabilistic models, and set $F_1^{\text{Top}} = F_1^{\text{bbox}}$ for TabPedia, for illustrative purposes. These models are labels in *italics*.

¹²<https://aws.amazon.com/ec2/pricing/on-demand/>

References

- [1] Abdelrahman Abdallah, Alexander Berendeyev, Islam Nuradin, and Daniyar Nurseitov. 2021. TNCR: Table Net Detection and Classification Dataset. *Neuro-computing* (2021). doi:10.1016/j.neucom.2021.11.101
- [2] Amihood Amir, Tzvika Hartman, Oren Kapah, Braha Shalom, and Dekel Tsur. 2007. Generalized LCS. *Theoretical Computer Science* 409, 50–61. doi:10.1007/978-3-540-75530-2_5
- [3] Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, Fabian Lindlbauer, Kasper Dinkla, Lokesh Mishra, Yusik Kim, Shubham Gupta, Rafael Teixeira de Lima, Valery Weber, Lucas Morin, Ingmar Meijer, Viktor Kuropiatnyk, and Peter W. J. Staar. 2024. Docling Technical Report. arXiv:2408.09869 [cs.CL]
- [4] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Iai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. arXiv:2502.13923 [cs.CV]
- [5] Muhammad Imam Luthfi Balaka, David Alexander, Qiming Wang, Yue Gong, Adila Krisnadhi, and Raul Castro Fernandez. 2025. Pnuma: Leveraging LLMs for Tabular Data Representation and Retrieval. *Proc. ACM Manag. Data* 3, 3 (2025), 200:1–200:28.
- [6] Galal M. Binmakhshen and Sabri A. Mahmoud. 2019. Document Layout Analysis: A Comprehensive Survey. *ACM Comput. Surv.* 52, 6, Article 109 (Oct. 2019), 36 pages. doi:10.1145/3355610
- [7] ZhaoWei Cai and Nuno Vasconcelos. 2017. Cascade R-CNN: Delving into High Quality Object Detection. arXiv:1712.00726 [cs.CV]
- [8] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. End-to-End Object Detection with Transformers. arXiv:2005.12872 [cs.CV]
- [9] Surekha Chandran and Rangachar Kasturi. 1993. Structural recognition of tabulated data. In *Proceedings of 2nd International Conference on Document Analysis and Recognition (ICDAR'93)*. IEEE, 516–519.
- [10] Zewen Chi, Heyan Huang, Heng-Da Xu, Houjin Yu, Wanxuan Yin, and Xian-Ling Mao. 2019. Complicated Table Structure Recognition. *arXiv preprint arXiv:1908.04729* (2019).
- [11] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. <https://lmsys.org/blog/2023-03-30-vicuna/>
- [12] Gheorghe Comanici and et. al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. arXiv:2507.06261 [cs.CL]
- [13] Cheng Da, Chuwei Luo, Qi Zheng, and Cong Yao. 2023. Vision Grid Transformer for Document Layout Analysis. arXiv:2308.14978 [cs.CV]
- [14] Harsh Desai, Pratik Kayal, and Mayank Singh. 2021. TabLeX: A Benchmark Dataset for Structure and Content Information Extraction from Scientific Tables. In *Document Analysis and Recognition – ICDAR 2021: 16th International Conference, Lausanne, Switzerland, September 5–10, 2021, Proceedings, Part II* (Lausanne, Switzerland). Springer-Verlag, Berlin, Heidelberg, 554–569. doi:10.1007/978-3-030-86331-9_36
- [15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. arXiv:2010.11929 [cs.CV]
- [16] Miao Fan and Doo Soon Kim. 2015. Detecting Table Region in PDF Documents Using Distant Supervision. arXiv:1506.08891 [cs.CV]
- [17] Liangcai Gao, Yilun Huang, Hervé Déjean, Jean-Luc Meunier, Qinqin Yan, Yu Fang, Florian Kleber, and Eva Lang. 2019. ICDAR 2019 Competition on Table Detection and Recognition (cTDaR). In *2019 International Conference on Document Analysis and Recognition (ICDAR)*. 1510–1515. doi:10.1109/ICDAR.2019.00243
- [18] Basilios Gatos, Dimitrios Danatsas, Ioannis Pratikakis, and Stavros J. Perantonis. 2005. Automatic Table Detection in Document Images. In *Pattern Recognition and Data Mining*, Sameer Singh, Maneesha Singh, Chid Apte, and Petra Pernert (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 609–618.
- [19] Ross Girshick. 2015. Fast R-CNN. In *2015 IEEE International Conference on Computer Vision (ICCV)*. 1440–1448. doi:10.1109/ICCV.2015.169
- [20] Max Göbel, Tamir Hassan, Ermelinda Oro, and Giorgio Orsi. 2012. A methodology for evaluating algorithms for table understanding in PDF documents. In *Proceedings of the 2012 ACM Symposium on Document Engineering (Paris, France) (DocEng '12)*. Association for Computing Machinery, New York, NY, USA, 45–48. doi:10.1145/2361354.2361365
- [21] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. arXiv:1706.04599 [cs.LG]
- [22] Yuxiang Guo, Zhonghao Hu, Yuren Mao, Baihua Zheng, Yunjun Gao, and Mingwei Zhou. 2025. Birdie: Natural Language-Driven Table Discovery Using Differentiable Search Index. *ArXiv preprint abs/2504.21282* (2025).
- [23] Max Göbel, Tamir Hassan, Ermelinda Oro, and Giorgio Orsi. 2013. ICDAR 2013 Table Competition. In *2013 12th International Conference on Document Analysis and Recognition*. 1449–1453. doi:10.1109/ICDAR.2013.292
- [24] Jaekyu Ha, R.M. Haralick, and I.T. Phillips. 1995. Recursive X-Y cut using bounding boxes of connected components. In *Proceedings of 3rd International Conference on Document Analysis and Recognition*, Vol. 2. 952–955 vol.2. doi:10.1109/ICDAR.1995.602059
- [25] Leipeng Hao, Liangcai Gao, Xiaohan Yi, and Zhi Tang. 2016. A table detection method for pdf documents based on convolutional neural networks. In *2016 12th IAPR Workshop on Document Analysis Systems (DAS)*. IEEE, 287–292.
- [26] Gaurav Harit and Anukriti Bansal. 2012. Table detection in document images using header and trailer patterns. In *Proceedings of the Eighth Indian Conference on Computer Vision, Graphics and Image Processing (Mumbai, India) (ICVGIP '12)*. Association for Computing Machinery, New York, NY, USA, Article 62, 8 pages. doi:10.1145/2425333.2425395
- [27] Jiani Huang, Haihua Chen, Fengchang Yu, and Wei Lu. 2024. From detection to application: Recent advances in understanding scientific tables and figures. *Comput. Surveys* 56, 10 (2024), 1–39.
- [28] Katsuhiko Itonori. 1993. Table structure recognition based on textblock arrangement and ruled line position. In *Proceedings of 2nd International Conference on Document Analysis and Recognition (ICDAR'93)*. IEEE, 765–768.
- [29] MAC Akmal Jahan and Roshan G Ragel. 2014. Locating tables in scanned documents for reconstructing and republishing. In *7th International Conference on Information and Automation for Sustainability*. IEEE, 1–6.
- [30] Thotringam Kasar, Philippine Barlas, Sebastien Adam, Clément Chatelain, and Thierry Paquet. 2013. Learning to detect tables in scanned document images using line information. In *2013 12th international conference on document analysis and recognition*. IEEE, 1185–1189.
- [31] Aamod Khataiwada, Roece Shraga, and Renée J. Miller. 2026. Diverse Unionable Tuple Search: Novelty-Driven Discovery in Data Lakes. In *Proceedings 29th International Conference on Extending Database Technology, EDBT 2026, Tampere, Finland, March 24–27, 2026*, Wolfgang Lehner, Vanessa Braganholo, Kostas Stefanidis, Zheyang Zhang, Alexander Krause, and João Felipe Nicolaci Pimentel (Eds.). OpenProceedings.org, 42–55. doi:10.48786/EDBT.2026.04
- [32] Thomas Kieninger and Andreas Dengel. 1998. The T-Recs Table Recognition and Analysis System, Vol. 1655. 255–269. doi:10.1007/3-540-48172-9_21
- [33] Christos Koutras, Jiani Zhang, Xiao Qin, Chuan Lei, Vassilis N. Ioannidis, Christos Faloutsos, George Karypis, and Asterios Katsifodimos. 2025. OmniMatch: Joinability Discovery in Data Products. *Proc. VLDB Endow.* 18, 11 (2025), 4588–4601. <https://www.vldb.org/pvldb/vol18/p4588-koutras.pdf>
- [34] Fabian Kupperts, Jan Kronenberger, Amirhossein Shantia, and Anselm Haselhoff. 2020. Multivariate Confidence Calibration for Object Detection. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE. doi:10.1109/cvprw50498.2020.00171
- [35] Vladimir I. Levenshtein. 1965. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet physics. Doklady* 10 (1965), 707–710. <https://api.semanticscholar.org/CorpusID:60827152>
- [36] Junlong Li, Yiheng Xu, Tengchao Lv, Lei Cui, Cha Zhang, and Furu Wei. 2022. DiT: Self-supervised Pre-training for Document Image Transformer. arXiv:2203.02378 [cs.CV]
- [37] Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, Ming Zhou, and Zhoujun Li. 2019. TableBank: A Benchmark Dataset for Table Detection and Recognition. arXiv:1903.01949 [cs.CV]
- [38] Yanghao Li, Hanzi Mao, Ross Girshick, and Kaiming He. 2022. Exploring Plain Vision Transformer Backbones for Object Detection. In *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part IX* (Tel Aviv, Israel). Springer-Verlag, Berlin, Heidelberg, 280–296. doi:10.1007/978-3-031-20077-9_17
- [39] Zhang Li, Yuliang Liu, Qiang Liu, Zhiyin Ma, Ziyang Zhang, Shuo Zhang, Zidun Guo, Jiarui Zhang, Xinyu Wang, and Xiang Bai. 2025. MonkeyOCR: Document Parsing with a Structure-Recognition-Relation Triplet Paradigm. arXiv:2506.05218 [cs.CV]
- [40] Tsung-Yi Lin, Michael Maire, Serge J. Belongie, Lubomir D. Bourdev, Ross B. Girshick, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. 2014. Microsoft COCO: Common Objects in Context. *CoRR abs/1405.0312* (2014). arXiv:1405.0312 <http://arxiv.org/abs/1405.0312>
- [41] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2018. Focal Loss for Dense Object Detection. arXiv:1708.02002 [cs.CV]
- [42] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- [43] Patrice Lopez. 2009. GROBID: Combining Automatic Bibliographic Data Recognition and Term Extraction for Scholarship Publications. In *Research and Advanced Technology for Digital Libraries*, Maristella Agosti, José Borbinha, Sarantos Kapidakis, Christos Papatheodorou, and Giannis Tsakonas (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 473–474.

- [44] Maksym Lysak, Ahmed Nassar, Nikolaos Livathinos, Christoph Auer, and Peter Staar. 2023. Optimized Table Tokenization for Table Structure Recognition. In *Document Analysis and Recognition - ICDAR 2023*, Gernot A. Fink, Rajiv Jain, Koichi Kise, and Richard Zanibbi (Eds.). Springer Nature Switzerland, Cham, 37–50.
- [45] Chixiang Ma, Weihong Lin, Lei Sun, and Qiang Huo. 2023. Robust Table Detection and Structure Recognition from Heterogeneous Document Images. *Pattern Recognition* 133 (Jan. 2023), 109006. doi:10.1016/j.patcog.2022.109006
- [46] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. 2015. Obtaining well calibrated probabilities using bayesian binning. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence* (Austin, Texas) (AAAI'15). AAAI Press, 2901–2907.
- [47] Ahmed Nassar, Nikolaos Livathinos, Maksym Lysak, and Peter Staar. 2022. TableFormer: Table Structure Understanding With Transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 4614–4623. doi:10.1109/CVPR52688.2022.00457
- [48] Alexandru Niculescu-Mizil and Rich Caruana. 2005. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning* (Bonn, Germany) (ICML '05). Association for Computing Machinery, New York, NY, USA, 625–632. doi:10.1145/1102351.1102430
- [49] OpenAI. 2025. *Introducing GPT-4.1 in the API*. Retrieved Jan 19, 2026 from <https://openai.com/index/gpt-4-1/>
- [50] Linke Ouyang, Yuan Qu, Hongbin Zhou, Jiawei Zhu, Rui Zhang, Qunshu Lin, Bin Wang, Zhiyuan Zhao, Man Jiang, Xiaomeng Zhao, Jin Shi, Fan Wu, Pei Chu, Minghao Liu, Zhenxiang Li, Chao Xu, Bo Zhang, Botian Shi, Zhongying Tu, and Conghui He. 2025. OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 24838–24848. doi:10.1109/CVPR52734.2025.02313
- [51] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a Method for Automatic Evaluation of Machine Translation. In *Annual Meeting of the Association for Computational Linguistics*. <https://api.semanticscholar.org/CorpusID:11080756>
- [52] Norman W. Paton, Jiaoyan Chen, and Zhenyu Wu. 2024. Dataset Discovery and Exploration: A Survey. *ACM Comput. Surv.* 56, 4 (2024), 102:1–102:37. doi:10.1145/3626521
- [53] Mateusz Pawlik and Nikolaus Augsten. 2016. Tree edit distance: Robust and memory-efficient. *Information Systems* 56 (2016), 157–173. doi:10.1016/j.is.2015.08.004
- [54] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Average precision score. *Journal of Machine Learning Research* 12 (2011), 2825–2830. https://scikit-learn.org/1.5/modules/generated/sklearn.metrics.average_precision_score.html
- [55] Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar. 2022. DocLayNet: A Large Human-Annotated Dataset for Document-Layout Segmentation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Washington DC, USA) (KDD '22). Association for Computing Machinery, New York, NY, USA, 3743–3751. doi:10.1145/3534678.3539043
- [56] PMC Open Access Subset [Internet]. 2003. . Bethesda (MD): National Library of Medicine. <https://pmc.ncbi.nlm.nih.gov/tools/openftlist/>
- [57] P. Pyreddy and W. B. Croft. 1997. *TINTI: A System for Retrieval in Text Tables TITLE2*. Technical Report. USA.
- [58] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 8748–8763. <https://proceedings.mlr.press/v139/radford21a.html>
- [59] Long Rujiao, Wang Wen, Xue Nan, Gao Feiyu, Yang Zhibo, Wang Yongpan, and Xia Gui-Song. 2021. Parsing Table Structures in the Wild. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- [60] Mahmoud Salaheldin Kasem, Abdelrahman Abdallah, Alexander Berendeyev, Ebrahim Elkady, Mohamed Mahmoud, Mahmoud Abdalla, Mohamed Hamada, Sebastiano Vascon, Daniyar Nurseitov, and Islam Taj-Eddin. 2024. Deep Learning for Table Detection and Structure Recognition: A Survey. *ACM Comput. Surv.* 56, 12, Article 305 (Oct. 2024), 41 pages. doi:10.1145/3657281
- [61] Sebastian Schreiber, Stefan Agne, Ivo Wolf, Andreas Dengel, and Sheraz Ahmed. 2017. DeepDeSRT: Deep Learning for Detection and Structure Recognition of Tables in Document Images. In *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, Vol. 01. 1162–1167. doi:10.1109/ICDAR.2017.192
- [62] Jeremy Singer-Vine and The pdfplumber contributors. 2025. *pdfplumber*. <https://github.com/jsvine/pdfplumber>
- [63] Brandon Smock, Rohith Pesala, and Robin Abraham. 2022. PubTables-1M: Towards Comprehensive Table Extraction From Unstructured Documents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 4634–4642.
- [64] Brandon Smock, Rohith Pesala, and Robin Abraham. 2023. Aligning Benchmark Datasets for Table Structure Recognition. In *Document Analysis and Recognition - ICDAR 2023*, Gernot A. Fink, Rajiv Jain, Koichi Kise, and Richard Zanibbi (Eds.). Springer Nature Switzerland, Cham, 371–386.
- [65] Brandon Smock, Rohith Pesala, and Robin Abraham. 2023. GriTS: Grid table similarity metric for table structure recognition. arXiv:2203.12555 [cs.LG]
- [66] Zhi Tian, Chunhua Shen, Hao Chen, and Tong He. 2019. FCOS: Fully Convolutional One-Stage Object Detection. arXiv:1904.01355 [cs.CV]
- [67] Yalin Wangt, Hsin T Phillipst, and Robert Haralick. 2001. Automatic table ground truth generation and a background-analysis-based table structure extraction method. In *Proceedings of Sixth International Conference on Document Analysis and Recognition*. IEEE, 528–532.
- [68] Haoran Wei, Chenglong Liu, Jinyue Chen, Jia Wang, Lingyu Kong, Yanming Xu, Zheng Ge, Liang Zhao, Jianjian Sun, Yuang Peng, et al. 2024. General OCR Theory: Towards OCR-2.0 via a Unified End-to-end Model. *arXiv preprint arXiv:2409.01704* (2024).
- [69] Yuxin Wu, Alexander Kirillov, Francisco Massa, Wan-Yen Lo, and Ross Girshick. 2019. Detectron2. <https://github.com/facebookresearch/detectron2>.
- [70] Fan Yang, Lei Hu, Xinwu Liu, Shuangping Huang, and Zhenghui Gu. 2023. A large-scale dataset for end-to-end table recognition in the wild. *Scientific Data* 10, 1 (2023), 110.
- [71] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. 2022. DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection. arXiv:2203.03605 [cs.CV]
- [72] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. arXiv:2205.01068 [cs.CL]
- [73] Zhenrong Zhang, Pengfei Hu, Jiefeng Ma, Jun Du, Jianshu Zhang, Huihui Zhu, Baocai Yin, Bing Yin, and Cong Liu. 2023. SEMv2: Table Separation Line Detection Based on Conditional Convolution. arXiv:2303.04384 [cs.CV]
- [74] Weichao Zhao, Hao Feng, Qi Liu, Jinqun Tang, Shu Wei, Binghong Wu, Lei Liao, Yongjie Ye, Hao Liu, Wengang Zhou, Houqiang Li, and Can Huang. 2024. TabPedia: towards comprehensive visual table understanding with concept synergy. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '24). Curran Associates Inc., Red Hook, NY, USA, Article 230, 28 pages.
- [75] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. 2024. DETRs Beat YOLOs on Real-time Object Detection. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 16965–16974. doi:10.1109/CVPR52733.2024.01605
- [76] Zhiyuan Zhao, Hengrui Kang, Bin Wang, and Conghui He. 2024. DocLayout-YOLO: Enhancing Document Layout Analysis through Diverse Synthetic Data and Global-to-Local Adaptive Perception. arXiv:2410.12628 [cs.CV]
- [77] Xinyi Zheng, Douglas Burdick, Lucian Popa, Xu Zhong, and Nancy Xin Ru Wang. 2021. Global Table Extractor (GTE): A Framework for Joint Table Identification and Cell Structure Recognition Using Visual Context. In *2021 IEEE Winter Conference on Applications of Computer Vision (WACV)*. 697–706. doi:10.1109/WACV48630.2021.00074
- [78] Xu Zhong, Elaheh ShafieiBavani, and Antonio Jimeno Yepes. 2020. Image-based table recognition: data, model, and evaluation. arXiv:1911.10683 [cs.CV]
- [79] Xu Zhong, Jianbin Tang, and Antonio Jimeno Yepes. 2019. PubLayNet: largest dataset ever for document layout analysis. In *2019 International Conference on Document Analysis and Recognition (ICDAR)*. IEEE, 1015–1022. doi:10.1109/ICDAR.2019.00166
- [80] Xu Zhong, Jianbin Tang, and Antonio Jimeno Yepes. 2019. PubLayNet: largest dataset ever for document layout analysis. arXiv:1908.07836 [cs.CL]
- [81] Xingyi Zhou, Dequan Wang, and Philipp Krähenbühl. 2019. Objects as Points. arXiv:1904.07850 [cs.CV]
- [82] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. 2021. Deformable DETR: Deformable Transformers for End-to-End Object Detection. arXiv:2010.04159 [cs.CV]